The Razor Pages programming model was introduced in ASP.NET Core 2.0 as a way to build server-side rendered page-based websites. It builds on top of the ASP.NET Core infrastructure to provide a streamlined experience, using conventions where possible to reduce the amount of boilerplate code and configuration required. In this section we'll look at a more complex page model to better understand the overall design of Razor Pages.

In listing 13.4 you saw a simple Razor Page that didn't contain any logic; instead, it only rendered the associated Razor view. This pattern may be common if you're building a content-heavy marketing website, for example, but more commonly your Razor Pages will contain some logic, load data from a database, or use forms to allow users to submit information.

To give you more of a flavor of how typical Razor Pages work, in this section we look briefly at a slightly more complex Razor Page. This page is taken from a to-do list application and is used to display all the to-do items for a given category. We're not focusing on the HTML generation at this point, so the following listing shows only the `PageModel` code-behind file for the Razor Page.

Listing 13.5 A Razor Page for viewing all to-do items in a given category

```
public class CategoryModel : PageModel
{
    private readonly ToDoService _service;                ❶
    public CategoryModel(ToDoService service)             ❶
    {
        _service = service;
    }

    public ActionResult OnGet(string category)            ❷
    {
        Items = _service.GetItemsForCategory(category);   ❸
        return Page();                                    ❹
    }
```